1 File structure

The code is found in a folder called «Code». The file structure is as follows:

```
Code
|
|— main.py
|
|— utilities
|        |
|        |— storer.py
|        |— grid.py
|        |— fields.py
|        |— potentials.py
|        |— evolvers.py
|        |— collisions.py
|
|— additionalfunctions
|        |
|        |— acceleration.py
|        |— adaptivecells.py
|        |— auxiliarycollisions.py
|        |— constants.py
|        |— densitync.py
|        |— interpolations.py
|        |— parsing.py
|        |— poscorrector.py
|        |— storing.py
|        |— weights.py
|
|— initialconditionsc
|        |
|        |— random.py
|        |— centralsoliton.py
|        |— gaussianpackets.py
|
|— initialconditionsnc
|        |
|        |— random.py
|        |— normal.py
|        |— thermaltest.py
|        |— thermalharmonic.py
```

The file main.py corresponds to the executable file which runs the code. The directory utilities contains defintions of classes and functions that are called by the main file to run the simulations. The directory additionalfunctions corresponds to files with specific instructions used by the main and the files in utilities. The directory initialconditionsc contains the available initial conditions for the coherent part and in initialconditionsnc we found the available initial conditions for the non-coherent part.

In this document we will review the files and the way to run the simulations.

1.1 Main file

The main function will run the code taking parameters from a file called inifile.dat. To run the code, in a terminal run the following command:

```
python main.py -path path_where_inifile_is
```

The program will run and all the outputs will be generated in the same path indicated after `-path`, which is where inifile.dat must be located. For any different running, copy inifile.dat to the folder where the data for specific simulation will be stored and edit the parameters there.

1.2 Inifile

Having introducing the existence of inifile.dat, which is the field containing all the information that will be used in the simulation, it is important to understand this file.

Any line starting with # is a comment and it is not considered in the simulation.

The way to write the parameters is with the name of the parameter with an equal symbol and the value:

`parameter=value`

It is possible to have spaces between the equal and the value or the name of the parameter and the code recognises them.

There are fundamental parameters for the running of the code. The names of these parameters are:

1. `boxlength`
2. `sitesbylength`
3. `totalnumber`
4. `coherentfraction`

The box of simulation is such that all the sides of equal measure in a three dimensional setup, defined with the parameter `boxlength`.

The parameter `sitesbylength` defines the resolution of the grid, it gives the number of points in each line.

The parameter `totalnumber` corresponds to the total number of particles or atoms to be simulated, so it must be an integer bigger than zero.

The parameter `coherentfraction` determines which part of the total number is coherent or not. It is a number between 0 and 1, where 0 corresponds to have only incoherent elements and 1 to have only coherent part.

Then we must fix the initial conditions for the field. Depending on the choice of coherent fraction, parameters about the initial conditions for the coherent and incoherent part will be required.

For the coherent initial conditions we must write the parameter `initialcondc`. The available initial conditions are:

1. random : Corresponds to set the coherent field with random values in each position.
2. from-file : It assigns the values for the field from a saved .npy file. When this initial condition is chosen, we must give another parameter in the inifile:
    a) `initialfilec` : The value of this parameter is the path and name of the file with the extension (i.e. `initialfilec=../runnings/initialcoherent.npy`).
3. centralsoliton : It assigns to the coherent field the value

$$\psi(r) = \sqrt{\frac{1024 N_{\rm tot}(0.091)^{3/2}}{33\pi^2 r_c^3\Big(1+0.091\left(\frac{r}{r_c}\right)^2\Big)^8}}e^{-i\theta(r)}$$

where $\theta(r)$ is a random phase, so it starts with a random value in each position. It requires to define another parameter in the inifile:

   a) `radc` : This is a radius r_c where the number density drops to aproximately half its central value.

4. gaussianpackets: It assigns two gaussian in $3D$.

For the incoherent part, we write the parameter `initialcondnc`. The available initial conditions are:

1. random : This initial condition fixes the incoherent field with random values in each position inside the box. Also, it assigns to the velocities of the particles a random value between 0 and `maxvel`, which is a parameter to be specified later. Here the value 1 corresponds to the reference velocity.

2. from-file : It assigns the values for the field from a saved .npy file. When this initial condition is chosen, we must give another parameter in the inifile:

   a) `initialfilenc` : The value of this parameter is the path and name of the file with the extension (i.e. `initialfilec=../runnings/initialincoherent.npy`).

3. normal : It set the positions and velocities of the incoherent part following a normal distribution: $\frac{1}{\sqrt{2\pi\sigma^2}}e^{-\frac{(x-\mu)^2}{2\sigma^2}}$. It requires four parameters:

   a) mupos : it represents the peak value of positions

   b) sigmapos : it defines how spread is the distribution of positions around the parameter mupos.

   c) muvel : it represents the peak value of velocities

   d) sigmavel : it defines how spread is the distribution of velocities around the parameter muvel.

4. thermalharmonic: It sets positions and velocities following a normal distribution as $e^{-\frac{1}{2}\bar{\beta}\bar{\omega}^2\bar{x}^2}e^{-\frac{1}{2}\bar{\beta}\bar{v}^2}$. It requires a parameter called

   a) beta: It is related with the associated temperature.

5. thermaltest: It sets positions randomly in a small box of length $L/32$. The velocities are randomly distributed with uniform distribution giving values between 0 and `maxvel`.

In the coherent and incoherent case also exists internally the initial condition `None`, which just indicates that the field associated is zero. When the coherent fraction $f=1$, immediately the initial condition for the incoherent part is set to `None`. In the case $f=0$, the coherent part is set automatically to `None`.

If the coherent fraction is smaller than 1 ($f<1$), then, it is required the parameters

1. `testparticlenumber` : It defines the number of test particles from the incoherent density to do the N-Body simulation.

2. `maxvel` : This parameter defines the maximum value of velocity (in reference units) that can be assigned.

Also, we need to specify the potential parameters. We have:

`harmonicpotential`

`contactinteraction`

To allow the existence of them we put this name of parameters with `=True`. If we don't put any, it is fixed to be `False` by default. When we allow some potentials, we need to add required parameters. For harmonic potential

$$V = \frac{1}{2}(\omega_x^2(x-x_0)^2 + \omega_y^2(y-y_0)^2 + \omega_z^2(z-z_0)^2)$$

we need:

`omegax`, `omegay`, `omegaz` : They parameterizes the values of the angular frequency ω_x, ω_y and ω_z in the harmonic potential.

`vxcenter`, `vycenter`, `vzcenter` : They parameterizes the values x_0, y_0 and z_0, where is the center of the harmonical trap.

For the contact interaction we need:

`selfcoupling` : It parameterizes the values of the coupling g for the contact interaction.

In addition, we need to specify some parameters for the evolution:

1. `iterations` : It is the number of steps of the evolution.

2. `recordevery` : It defines the number of steps when the code saves the data of the evolution. This number must be smaller or the same as iterations.

3. `imagprop` : It is a boolean parameter. Only accepts True or False. If True, the simulation will run imaginary time propagation. If False, the simulation will proceed with real time propagation.

4. `factordt`: It is an int type variable. This value will be a factor 10^{-x} with x the value in the file which multiply the step dt.

There are some optional parameters that if there are not added they are set `False` by default. The parameter `smoothing`, defines if we will convolve the coherent and incoherent densities to smooth the possible spikes appearing from the discrete consideration in the incoherent part and the noise appearing in the coherent sector. Thus, it can be True or False. If it is True, it is required a parameter `smoothfactor`, which is the value of σ in a gaussian filter.

1.3 Utilities

Here we will describe elements about the files in the folder utilities, which are called by the main.

1.3.1 storer.py

In order to generate the simulation, the program extracts the information from the file inifile.dat and it stores that information in an array.

The only function defined in this file is «reader»:

```
def reader(filepath)
```

The first argument, `filepath`, is the path where inifile.dat will be readed.

The first definitions in the function `reader` are five arrays containing strings:

1. `fundparameterlist`
2. `icclist`
3. `icnclist`
4. `potentiallist`
5. `optionalflags`

The first array, `parameterlist` is defined by:

```
parameterlist=['boxlength','sitesbylength','totalnumber',
'coherentfraction']
```

This array contains the names of the fundamental parameters in inifile.dat. They will be used to check if they are present, otherwise, the code won't run and it will give a warning that one of them is missing and you must add it.

The next two arrays of strings are `icclist` and `icnclist`, which contain sub-arrays for the available initial conditions of the coherent and incoherent part. For both cases, the array is

```
[['None'],['testing'],['random'],['otherinitialcondition','parameter1',...],...]
```

Every sub-array corresponds to a specific initial condition. The first three are common to `icclist` and `icnclist` and the rest, depending on the type, always has the same structure. The first entry is the name of the initial condition and the next entries are the needed parameters for the corresponding initial condition.

The array `potentiallist` contains the parameters to construct the potential for the coherent and incoherent parts. This array is composed by

```
potentiallist=[['harmonicpotential','omegax','omegay','omegaz'],
['contactinteraction','selfcoupling']]
```

The array

```
optionalflags=['smoothing']
```

contains the optional options that the code admits.

Then, the function open the file, remove empty lines and comments and it creates five arrays called `fundparameters`, `fieldparameters`, `potparameters`, `evolverparameters` and `optparameters` where the parameters will be stored. The elements of each array are:

- `fundparameters`

→ `fundparameters[0]`: Length of the simulation box (all the sides are the same). It is a `float` type.

→ `fundparameters[1]`: Number of sites by length. It is an `int` type.

→ `fundparameters[2]`: Total number of elements, coherent and incoherent. It is an `int` type.

→ `fundparameters[3]`: Coherent fraction of the system. It is a `float` type.

→ `fundparameters[4]`: It contains the information on the path to save files.

- `fieldparameters`

→ `fieldparameters[0]`: Array for the initial conditions for coherent part.

  - `fieldparameters[0][0]`: Name of the initial condition for the coherent part. It is a `string` type.

  - `fieldparameters[0][i]`: Parameters of the initial conditions. It is a `string` type only if the initial condition is «from-file», indicating the path of the file. Any other case, it is a `float` type.

→ `fieldparameters[1]`: Array for the initial conditions for incoherent part.

  - `fieldparameters[1][0]`: Name of the initial condition for the incoherent part. It is a `string` type.

  - `fieldparameters[1][i]`: Parameters of the initial conditions. It is a `string` type only if the initial condition is «from-file», indicating the path of the file. Any other case, it is a `float` type.

→ `fieldparameters[2]`: Array containing information about the incoherent particles.

  - `fieldparameters[2][0]`: Number of the test particles. It is an `int` type.

  - `fieldparameters[2][1]`: Maximum value of the velocities of the incoherent part. It is a `float` type.

- `potparameters`

→ `potparameters[0]`: Parameters related to the harmonical potential.

  - `potparameters[0][0]`: Parameter harmonicpotential. It is a `boolean` type defining if it will be present or not.

  - `potparameters[0][1]`: Parameter omegax. It is a `float` type.

  - `potparameters[0][2]`: Parameter omegay. It is a `float` type.

  - `potparameters[0][3]`: Parameter omegaz. It is a `float` type.

  - `potparameters[0][4]`: Parameter vxcenter. It is a `float` type.

  - `potparameters[0][5]`: Parameter vycenter. It is a `float` type.

  - `potparameters[0][6]`: Parameter vzcenter. It is a `float` type.

→ `potparameters[1]`: Parameters related to the contact interaction.

  - `potparameters[1][0]`: Parameter contactinteraction. It is a `boolean` type defining if it will be present or not.

  - `potparameters[1][1]`: Parameter g related to the self-interaction. It is a `float` type.

- `evolverparameters`

→ `evolverparameters[0]`: Number of step iterations. It is an `int` type.

→ `evolverparameters[1]`: Number of steps to save the field. It is an `int` type.

→ `evolverparameters[2]`: Indicates if the propagation is in imaginary time. It is a `boolean` type. True if it is imaginary time and False is it is in real time.

→ `evolverparameters[3]`: Factor that appears as $10^{-\text{evolverparameters[3]}}$ to multiply the step dt. It is an `int` type.

- `optparameters`

→ `optparameters[0]`: It relates to the smoothing.

  – `optparameters[0][0]`: Indicates if there is smoothing. It is a `boolean` type. True if there is smoothing and False if there is not.

  – `optparameters[0][1]`: Contains the value of the sigma of the gaussian filter defined in the inifile as `smoothfactor` `optparameters[0][0]` is True. It is a `float` type.

1.3.2 grid.py

This file contains the instruction to create the grid in the simulation box. The only function defined here is

```
def creategrid(fundparameters)
```

The argument will be the array called parameters created from the instructions in storer.py. This function will require the use of Fast Fourier Transform (FFT) to create a momentum grid together with the spatial grid. The implementation of FFT here is used with the library scipy.fft.

Here we need to specify the discretization of this implementation to clarify later uses in the code, since it will be used continuously. To discretize in $1D$ (generalization to the case 3D used in the code is straightforward):

$$\tilde{f}(k) = \int_{-\infty}^{\infty} dx\, f(x) e^{-ikx} \to \sum_{n=0}^{N-1} \Delta x\, f(n\Delta x) e^{-ikn\Delta x}$$

Notice that here to pass to the continuous we have truncated the integral in a certain interval, which we will consider $\left[-\frac{L}{2}, \frac{L}{2}\right)$, for a certain length L, which will correspond to the box length (`parameters[1]`), and we have chosen N sites per length, defining the space between the N sites in the length as $\Delta x = \frac{L}{N}$. Since we have chosen N spatial sites, we will also use N equally spaced sites for momenta: $k = \frac{2\pi m}{L}$. We observe that n is related with the discrete index for x and m will be related with a discrete index for k. With that, we have that the integral now can be written as

$$\tilde{f}(m) = \Delta x \sum_{n=0}^{N-1} f(n) e^{-\frac{2\pi i n m}{N}}$$

where we have changed $n\Delta x \to n$ in the argument of f since Δx is a constant and we changed the argument in $\tilde{f}(k)$ for m which is the index corresponding to k.

Same can be done for the inverse and it defines:

$$f(x) = \frac{1}{2\pi}\int_{-\infty}^{\infty} dk\, \tilde{f}(k)e^{ikx} \to f(n) = \frac{1}{L}\sum_{m=0}^{N-1} \tilde{f}(m)e^{i\frac{2\pi mn}{N}}$$

If we do the change $\tilde{f}(m) \to \Delta x \tilde{f}(m) = \frac{L}{N}\tilde{f}(m)$ we will have:

$$\tilde{f}(m) = \sum_{n=0}^{N-1} f(n)e^{-\frac{2\pi inm}{N}}$$

$$f(n) = \frac{1}{N}\sum_{m=0}^{N-1} \tilde{f}(m)e^{i\frac{2\pi mn}{N}}$$

which are the objects that are used in the routines of FFT in the scipy library.

Coming back to the function `creategrid`, it will build a spatial grid defined through the interval of length L (`fundparameters[0]`) with N points (`fundparameters[1]`) as $\left[-\frac{L}{2}, \frac{L}{2}\right)$ constructed in the array `xlin` inside this function. We don't consider the endpoint, indicating it in the function `np.linspace` with the option `endpoint=False`, to work with periodical boundary conditions. As we mentioned in the discretization for FFT, the space between sites will be denoted inside the function as `dx` and it is defined by `dx=fundparameters[0]/fundparameters[1]`.

The momentum grid is built through the definition of $k = \frac{2\pi m}{L}$. This is implemented using the function `fftfreq(parameters[1],dx)` which returns $\frac{m}{L}$ in the order:

`[0,1,...,fundparameters[1]/2-1,-fundparameters[1]/2,...,-1]/(dx*fundparameters[1])` for m even

`[0,1,...,(fundparameters[1]-1)/2,-(fundparameters[1]-1)/2,...,-1]`
`/(dx*fundparameters[1])` for m odd

For this reason, in the array `klin` these numbers are multiplied by 2π.

Using `np.meshgrid` the full grid is constructed from `xlin` and `klin` where the option `copy=False` helps to save memory avoiding innecesary copies of the elements of the arrays.

The function returns the following parameters:

[0]: It corresponds to an array of momenta squared k^2

[1]: It corresponds to an array of $1/k^2$, being 0 if $k = 0$.

[2]: It corresponds to an array for the x component of the spatial grid

[3]: It corresponds to an array for the x component of the momentum grid

[4]: It corresponds to an array for the y component of the spatial grid.

[5]: It corresponds to an array for the y component of the momentum grid.

[6]: It corresponds to an array for the z component of the spatial grid.

[7]: It corresponds to an array for the z component of the momentum grid.

1.3.3 fields.py

This file contains the information to create the configurations of the objects to be simulated. To do this, the code defines a class: `class boson`, which is composed of two arrays for the coherent and quasiparticles and its properties.

This class is initialized with

```
def __init__(self, fundparameters, fieldparameters, potparameters, optparameters,
grids):
```

where the initialization starts storing elements required during the simulations:

```
self.dx=cts.FTYPE(fundparameters[0]/fundparameters[1])
```

```
self.totalnumber=fundparameters[2]
```

```
self.cohfrac=fundparameters[3]
```

```
self.tpnumber=fieldparameters[2][0]
```

```
self.smooth=optparameters[0][1] if optparameters[0][0] else None
```

```
self.unigrid=grids[6][0][0]
```

```
self.g=potparameters[1][1]
```

Also, a cache will be stored to avoid recalculate quantities. We initialize it as `None`:

```
self._cache_densityc    = None
```

```
self._cache_densitydep    = None
```

```
self._cache_interp_dens = None
```

```
self._cache_densityq    = None
```

```
self._cache_thermal     = None
```

Then, the coherent part is created using first the instruction

```
grid_shape=(fundparameters[1],)*3
```

to define an object containing the number of sites per length `fundparameters[1]`. The result of `grid_shape` is for example `(128, 128, 128)` for `fundparameters[1]=128`.

With it, according to the initial conditions it generates a complex array called `self.fieldc`.

The coherent objects contains a real and imaginary part defined by `self.re` and `self.imag` respectively.

Also, the coherent part is initially normalized using a function called `normalizer` also defined in this file.

After this, the test particles for quasiparticles are created in:

```
self.fieldnc=np.zeros((2,fieldparameters[2][0],3))
```

where the first argument is for positions and velocities of the test particles. The value [0] corresponds to the positions and [1] for velocities. The second argument, `fieldparameters[2][0]`, is the number of test particles that will be used in the simulation. The last argument is the dimension. It means that in $3D$ the array for 4 test particles will be

```
array([[[0., 0., 0.],
        [0., 0., 0.],
        [0., 0., 0.],
        [0., 0., 0.]],
```

```
       [[0., 0., 0.],
        [0., 0., 0.],
        [0., 0., 0.],
        [0., 0., 0.]]])
```

After creating this array, it is filled according to the given initial conditions.

Also, an array

```
self.x0buffer = np.zeros_like(self.fieldnc[0], dtype=cts.FTYPE)
```

is defined to be used in the evolver for the predictor-corrector part during the evolution. With this, the initialization is closed.

After the initialization the file contains all the functions of the class. First, we have functions to invalidate the stored cache during the evolution:

```
def invalidate_fieldc_cache(self):
        self._cache_densityc    = None
        self._cache_densitydep  = None
        self._cache_interp_dens = None
        self._cache_densityq    = None
        self._cache_thermal     = None
def _invalidate_phase_only(self):
        pass
def invalidate_fieldnc_cache(self):
        self._cache_interp_dens = None
        self._cache_densityq    = None
        self._cache_thermal     = None
def invalidate_positions_only(self):
        self._cache_interp_dens = None
def invalidate_for_potential(self):
        self._cache_densityq    = None
        self._cache_thermal     = None
```

Also, the class contains properties that will be used in some functions in the code. First, the function to normalize the field, which was called in the initialization:

```
def normalizer(self):
        Ntarget = self.cohfrac * self.totalnumber
        diff=Ntarget
        if Ntarget!=0:
            while abs(diff) / Ntarget > 1e-6:
                Nc0   = np.sum(self.densityc)   * self.dx**3
```

```
        Ndep0 = np.sum(self.densitydep) * self.dx**3


        diff = Ntarget - (Nc0 + Ndep0)
        denom = 2.0 * Nc0 + 3.0 * Ndep0


        if denom != 0:
            delta = diff / denom
            self.fieldc *= (1.0 + delta)
            self.invalidate_fieldc_cache()
```

Here, the total number must be preserved so, the factor to multiply the field $(1+\delta)$ to obtain such normalization is obtained through solving the cubic equation that appears from

$$\int d^3x \left(n_c + \text{factor} \cdot n_c^{3/2}\right) = fN_{\text{tot}}$$

The next functions, are functions to get the values of stored quantities that are important for the class:

1. `def getdx(self)` : This function calls the space between grid points.
2. `def getunigrid(self)` : This function returns the pattern $1D$ grid that generates the full grid
3. `def gettotalnumber(self)` : This function calls the total number of particles.
4. `def getcohfrac(self)` : This function returns the value of coherent fraction.
5. `def gettpnumber(self)`: This function returns the value of the test particles.

The next group of properties are the density and phase of the coherent part, defined as:

```
@property
def densityc(self):
        if self._cache_densityc is None:
            dens = np.abs(self.fieldc)**2
            if self.smooth is not None:
                dens = gaussian_filter(dens, sigma=self.smooth)
            self._cache_densityc = dens.astype(cts.FTYPE)
        return self._cache_densityc
@property
def phase(self):
    return np.angle(self.fieldc)
```

Here the density of the coherent part is basically $|\Phi|^2$ and we take a gaussian filter to smooth the density if `optparameters[0][0]` is True.

The phase of the coherent complex field is computed with the numpy function `angle`, which returns the angle of a complex number or array of complex numbers as in this case.

Also, we have a part of the total density coming from the depletion term $\frac{1}{3\pi^2}\frac{(mg)^{3/2}}{\hbar^3}n_c^{3/2}$ defined by

```
@property
def densitydep(self):
        if self._cache_densitydep is None:
            dep = ((self.g**1.5) * (self.densityc**1.5)) / (3*np.pi**2)
            if self.smooth is not None:
                dep = gaussian_filter(dep, sigma=self.smooth)
            self._cache_densitydep = dep.astype(cts.FTYPE)
        return self._cache_densitydep
```

In some parts of the evolution will be required the value of the coherent density interpolated to the points where the non-coherent test particles are. To do that, it is defined the function:

```
def getinterpdens(self):
        if self._cache_interp_dens is None:
            self._cache_interp_dens = interp.interpolatordens(self)
        return self._cache_interp_dens
```

To make this function work, we will employ Numba, which will be called in the auxiliary function `interpolatordens`, which is in the folder additionalfunctions in interpolations.py.

In order to compute the density of the quasiparticles in the non-coherent part and the thermal contribution to the coherent equation, we define two weight functions, which will call functions from an auxiliary file: weights.py. This file computes in Numba these weights. The first of them is called `weight1` which computes the weight

$$W_1 = \frac{\bar{v}_i^2 + 2\bar{g}\bar{n}_c(\bar{x})}{\sqrt{\bar{v}_i^2(\bar{v}_i^2 + 4\bar{g}\bar{n}_c(\bar{x}))}}$$

and it is defined by

```
def weight1(self):
        densterm = (self.g * self.getinterpdens()).astype(cts.FTYPE)
        modvelsq = np.sum(self.fieldnc[1] ** 2, axis=1).astype(cts.FTYPE)
        return wg.weight1numba(modvelsq, densterm)
```

The second function is `weight2` defined by

$$W_2 = \sqrt{\frac{\bar{v}_i^2}{\bar{v}_i^2 + 4\bar{g}\bar{n}_c}}$$

The quasiparticle density is defined by

```
def weight2(self):
        densterm = (self.g * self.getinterpdens()).astype(cts.FTYPE)
```

```
        modvelsq = np.sum(self.fieldnc[1] ** 2, axis=1).astype(cts.FTYPE)
        return wg.weight2numba(modvelsq, densterm)
```

The structure is similar to `weight1`, but with a different function. Both weights will be relevant also in the non-coherent evolution since they appear in the position and velocity equations respectively.

With these functions we can define a function `def computenqandthermal(self)` that computes simultaneosuly the functions:

$$\bar{\tilde{n}} = \frac{(1-f)N_{\rm tot}}{\Omega}\sum_{i=1}^{\tilde{N}_{\rm test}} \frac{\bar{v}_i^2 + 2\bar{g}\bar{n}_c(\bar{x})}{\sqrt{\bar{v}_i^2(\bar{v}_i^2 + 4\bar{g}\bar{n}_c(\bar{x}))}}\delta(\bar{x}-\bar{x}_i)$$

$$\bar{\tilde{\mathfrak{F}}}[f] = \frac{(1-f)N_{\rm tot}}{\Omega}\sum_{i=1}^{\tilde{N}_{\rm test}} \sqrt{\frac{\bar{v}_i^2}{\bar{v}_i^2 + 4\bar{g}\bar{n}_c}}\delta(\bar{x}-\bar{x}_i)$$

To compute these objects it is called a function `computedensity` from `additionalfunctions.densitync`, which uses Numba.

With this function, we define the the quasiparticle density and the thermal contribution with:

```
#Quasiparticle density
@property
def densityq(self):
        if self._cache_densityq is None:
            if self.getcohfrac() == 1:
                self._cache_densityq=np.zeros((len(self.unigrid),) * 3,
dtype=cts.FTYPE)
            else:
                self.computenqandthermal()
        return self._cache_densityq


#Thermal contribution
@property
def thermalcontribution(self):
        if self._cache_thermal is None:
            if self.getcohfrac() == 1:
                self._cache_thermal=np.zeros((len(self.unigrid),) * 3,
dtype=cts.FTYPE)
            else:
                self.computenqandthermal()
        return self._cache_thermal
```

1.3.4 potentials.py

This file is related to the construction of the arrays for the potential. The potentials are defined through a `class potential`. This class stores the effective contact interaction coupling in

```
self.geff=potparameters[1][1]
```

and initialize the harmonical potential, which is only computed once:

```
self.harmpot = self.buildharmonic(potparameters, grids)
```

through a function defined in the file called `buildharmonic`.

Then, three potentials are defined:

```
self.potentialc=self.buildpotentialc(fieldY)
```

```
self.mfpotential=self.buildmfpotential(fieldY)
```

```
self.potentialq=self.buildpotentialq(fieldY)
```

The first of them corresponds to the coherent potential defined by

$$\bar{V}_{\text{ext}} + \bar{g}\bar{n}_{\text{tot}} + \frac{4}{3\pi^2}\bar{g}^{5/2}\bar{n}_c^{3/2} + \bar{g}\tilde{\mathfrak{F}}[f]$$

The second, corresponds to the non-coherent mean-field potential

$$\bar{V}_{\text{ext}} + 2\bar{g}\bar{n}_{\text{tot}} - \bar{g}\bar{n}_c$$

and the last one, is just

$$\bar{g}\bar{n}_c$$

which is going inside the quasiparticle energy.

After that we initialize a value

```
self.potversion = 0
```

This value will change everytime that the potential is updated and it will be useful when the acceleration of the non-coherent elements will be computed.

With this, the initialization function is complete.

To construct the harmonical potential it uses the function `buildharmonic` defined by

```
def buildharmonic(self,potparameters, grids):
        coord_indices = (2, 4, 6)[:3]
        coords = [grids[i] for i in coord_indices if len(grids) > i]
        if len(coords) < 3:
            coords += [np.zeros_like(coords[0])] * (3 - len(coords))

        omegas = np.array(potparameters[0][1:4], dtype=cts.FTYPE)
        shifts = np.array(potparameters[0][4:7], dtype=cts.FTYPE)
        vharm=0.5 * sum((omegas[i]**2) * (coords[i]-shifts[i])**2 for i in
range(3))
```

```
        return (vharm).astype(cts.FTYPE)
```

To construct the coherent potential some functions are required: `contactc`, `contactq`, `contactdep`, `LHY` and `thermpot`, defined as

```
def contactc(self, fieldY):
        return (self.geff*fieldY.densityc).astype(cts.FTYPE)


def contactq(self, fieldY):
        return (self.geff*fieldY.densityq).astype(cts.FTYPE)


def contactdep(self, fieldY):
        return (self.geff*fieldY.densitydep).astype(cts.FTYPE)


def LHY(self, fieldY):
        coeff = 4.0 / (3.0 * np.pi**2)
        return (coeff*(self.geff**2.5)*fieldY.densityc**1.5).astype(cts.FTYPE)


def thermpot(self,fieldY):
        return (self.geff*fieldY.thermalcontribution).astype(cts.FTYPE)
```

With these auxiliary function, the function `buildpotentialc` is defined by:

```
def buildpotentialc(self, fieldY):
        if fieldY.getcohfrac() == 0:
            return np.zeros_like(fieldY.densityc, dtype=cts.FTYPE)
        g = self.geff
        nc, nq, ndep = fieldY.densityc, fieldY.densityq, fieldY.densitydep
        lhy = cts.FTYPE(4.0/(3.0*np.pi**2)) * (g**2.5) * nc**1.5
        return (self.harmpot + g*nc + g*nq + g*ndep + lhy +
g*fieldY.thermalcontribution).astype(cts.FTYPE)
```

Then, for the quasiparticle sector we have the two functions:

```
def buildmfpotential(self, fieldY):
        g = self.geff
        nc, nq, ndep = fieldY.densityc, fieldY.densityq, fieldY.densitydep
        return (self.harmpot + g*nc + 2*g*nq + 2*g*ndep).astype(cts.FTYPE)


def buildpotentialq(self, fieldY):
```

```
        if fieldY.getcohfrac() == 1:
            return np.zeros_like(fieldY.densityc, dtype=cts.FTYPE)
        return (self.geff * fieldY.densityc).astype(cts.FTYPE)
```

Also, we have a function `updatepotentials(self, fieldY, grids)` to update the potentials during the evolution, and a last function that allows to add an array as an extra potential to the coherent part:

```
def addextracontribution(self,cont):
        self.potentialc+=cont
        return None
```

1.3.5 evolvers.py

In this file we will define the functions needed for the method to do the temporal evolution. The implemented method is an splitting method for the coherent and non-coherent part.

For the coherent part, the idea is that always we have something like

$$\frac{\partial\bar{\Phi}_0}{\partial\bar{t}} = -i\left[-\frac{1}{2}\bar{\nabla}^2 + \bar{U}\right]\bar{\Phi}_0$$

This has as solution:

$$\bar{\Phi}_0(t+\Delta t) = e^{-i\left[-\frac{1}{2}\bar{\nabla}^2+\bar{U}\right]\Delta t}\bar{\Phi}_0(t)$$

Using the Baker-Hausdorff-Campbell formula for the exponential we can write

$$\bar{\Phi}_0(t+\Delta t) = e^{-i\bar{U}\frac{\Delta t}{2}}e^{-i\left[-\frac{1}{2}\bar{\nabla}^2\right]\Delta t}e^{-i\bar{U}\frac{\Delta t}{2}}\bar{\Phi}_0(t) + \mathcal{O}(\Delta t^3)$$

where the error of is of order Δt^3.

Thus, we do the evolution in three steps. The first, called kick, consists in the multiplication of the function by $e^{-i\bar{U}\frac{\Delta t}{2}}$. Then, as a second step, called drift, we apply $e^{-i\left[-\frac{1}{2}\bar{\nabla}^2\right]\Delta t}$ to the previous result. A fast way to do this is going to the Fourier space, where $e^{-i\left[-\frac{1}{2}\bar{\nabla}^2\right]\Delta t} \to e^{-i\left[\frac{1}{2}k^2\right]\Delta t}$. Then, we come back to the coordinate space to do the last step, another kick. That closes the evolution during one timestep. Summarising:

$$\bar{\Phi}_0(t+\Delta t) = e^{-i\bar{U}\frac{\Delta t}{2}}\mathrm{IFFT}\left(e^{-i\left[\frac{1}{2}\bar{k}^2\right]\Delta t}\mathrm{FFT}\left(e^{-i\bar{U}\frac{\Delta t}{2}}\bar{\Phi}_0(t)\right)\right) + \mathcal{O}(\Delta t^3)$$

For the non-coherent part, since the equations for evolution:

$$\frac{\partial\bar{x}_i}{\partial\bar{t}} = \frac{\bar{v}_i^2 + 2\bar{g}\bar{n}_c}{\sqrt{\bar{v}_i^2(\bar{v}_i^2+4\bar{g}\bar{n}_c)}}\bar{v}_i,\ \frac{\partial\bar{v}_i}{\partial\bar{t}} = -\sqrt{\frac{\bar{v}_i^2}{\bar{v}_i^2+4\bar{g}\bar{n}_c}}\bar{g}\bar{\nabla}\bar{n}_c - \bar{\nabla}(\bar{V}_{\mathrm{ext}} + 2\bar{g}\bar{n}_{\mathrm{tot}} - \bar{g}\bar{n}_c)$$

is non-separable, the scheme is not symplectic. It means that there could be drifts from the total number conservation and energy. A small timestep helps to prevent a dramatic drift. Also, to contribute to the stability we use for this case the splitting method with a predictor-corrector scheme. First, we do a kick, updating the velocities according to:

$$\bar{v}_i(t+\Delta t) = \bar{v}_i(t) + (-W_2(\bar{x}_i(t),t)\bar{g}\bar{\nabla}\bar{n}_c - \bar{\nabla}(\bar{V}_{\mathrm{ext}} + 2\bar{g}\bar{n}_{\mathrm{tot}}(\bar{x}_i) - \bar{g}\bar{n}_c(\bar{x}_i)))\Delta t$$

with $W_2(\bar{x}_i(t),t) = \sqrt{\frac{\bar{v}_i(t)^2}{\bar{v}_i(t)^2 + 4\bar{g}\bar{n}_c(\bar{x}_i)}}$

then, we do a drift, updating the positions with the predictor-corrector scheme:

$$\bar{x}_i^{\text{pred}} = \bar{x}_i(t) + W_1(\bar{x}_i(t),t)\bar{v}_i(t)\Delta t$$

$$\bar{x}_i^{\text{corr}} = \bar{x}_i(t) + W_1\left(\frac{\bar{x}_i(t)+\bar{x}_i^{\text{pred}}}{2}, t\right)\bar{v}_i(t)\Delta t$$

$$\bar{x}_i(t+\Delta t) = \bar{x}_i(t) + W_1\left(\frac{\bar{x}_i(t)+\bar{x}_i^{\text{corr}}}{2}, t\right)\bar{v}_i(t)\Delta t$$

with $W_1(\bar{x}_i(t),t) = \frac{\bar{v}_i(t)^2 + 2\bar{g}\bar{n}_c(\bar{x}_i)}{\sqrt{\bar{v}_i(t)^2(\bar{v}_i(t)^2 + 4\bar{g}\bar{n}_c(\bar{x}_i))}}$

and then, a kick again.

With that, we define the first function, which is the core of the evolution:

```
def evolver(fundparameters,fieldparameters,evolverparameters, grids, fieldY, pot):
```

The first action of the function is to define

```
chi=1.0 if evolverparameters[2] else 1.0j
```

where the value 1 corresponds to imaginary evolution or i if it is in real time evolution.

Then, we construct the arrays to store the time and the rate of collisions:

```
num_store = evolverparameters[0] // evolverparameters[1] + 1
```

```
timeline=np.zeros(num_store, dtype=cts.FTYPE)
```

```
gammacol2 = np.zeros((len(fieldY.getunigrid()),) * 3, dtype=cts.FTYPE)
```

and set initial indices that will be updated during the evolution, together with $dx^2/6$ used for the adaptive time:

```
t=cts.FTYPE(0.0)
```

```
ncollsteps  = 0
```

```
storeidx = 0
```

```
dx2over6=(fieldY.getdx()**2)/6.0
```

then the initial configuration is stored with

```
sto.savingstep(storeidx, t, fundparameters, evolverparameters, fieldY, timeline,
gammacol2, ncollsteps)
```

which uses an auxiliary function.

After that, the function enters in the evolution loop with

```
for h in range(1,evolverparameters[0]+1):
```

There, the timesteps are adapted with

```
maxpot=cts.FTYPE(np.maximum(np.amax(np.abs(pot.potentialc))
,np.amax(np.abs(pot.potentialq))))
```

```
        if maxpot==0 or np.isnan(maxpot)==True:
```

```
            dt=cts.FTYPE(10**(-evolverparameters[3])*dx2over6)
        else:

dt=cts.FTYPE(10**(-evolverparameters[3])*np.minimum(dx2over6,1/maxpot))
```

Then, the process of kick-drift-kick is called with

```
kickdrift(chi,dt,fundparameters,grids,fieldY,pot)
```

and this function is defined in this file.

Then it normalizes the coherent field if the evolution is in imaginary time, since there the exponential application is not unitary.

After that, if there are non-coherent parts, and in real time, and the flag `nocollisions` is not present, the collisional part is called with

```
if fieldY.getcohfrac() < 1.0 and evolverparameters[2]==False and
optparameters[1][0]==False:
    collisionstep(evolverparameters,optparameters,grids,fieldY,dt,
gammacol2,ncollsteps)
```

which is a function in the same file.

Then, the time is updated: `t += dt`, and it saves to output files with

```
if h%(evolverparameters[1])==0:
            storeidx += 1
            sto.savingstep(storeidx, t, fundparameters, evolverparameters, fieldY,
timeline, gammacol2, ncollsteps)
```

This storing closes the function.

After the function `evolver`, it is defined the kick process:

```
def kick(chi, dt, fundparameters, grids, fieldY, pot):
    # Kick non-coherent
    if fieldY.getcohfrac() < 1 and len(fieldY.fieldnc[0]) > 0:
        w2 = fieldY.weight2()[:, None]
        acel_q, acel_mf = acceleration(grids, fieldY, pot)
        fieldY.fieldnc[1] += (acel_q * w2 + acel_mf) * dt
    # Kick coherent
    if fieldY.getcohfrac() > 0:
        np.multiply(fieldY.fieldc,np.exp(-chi * dt *
pot.potentialc),out=fieldY.fieldc)
        fieldY._invalidate_phase_only()
```

This function uses the function `acceleration` from `additionalfunctions.acceleration` to compute the part: $-W_2(\bar{x}_i(t),t)\bar{g}\bar{\nabla}\bar{n}_c - \bar{\nabla}(\bar{V}_{\text{ext}} + 2\bar{g}\bar{n}_{\text{tot}}(\bar{x}_i) - \bar{g}\bar{n}_c(\bar{x}_i))$.

Also, we define the drift process:

```
def drift(chi, dt, grids, fieldY, pot):

    #Drift for non-coherent

    if fieldY.getcohfrac() < 1 and len(fieldY.fieldnc[0]) > 0:

        np.copyto(fieldY.x0buffer, fieldY.fieldnc[0])

        v_base = fieldY.fieldnc[1]

        w1_0 = fieldY.weight1()[:, None]

        fieldY.fieldnc[0] += v_base * w1_0 * dt

        fieldY.invalidate_positions_only()

        for _ in range(2):

            fieldY.fieldnc[0] += fieldY.x0buffer

            fieldY.fieldnc[0] *= 0.5

            fieldY.fieldnc[0] = pcr.poscorrector(fieldY.fieldnc[0],
fieldY.getunigrid())

            fieldY.invalidate_positions_only()

            w1_mid = fieldY.weight1()[:, None]

            np.copyto(fieldY.fieldnc[0], fieldY.x0buffer)

            fieldY.fieldnc[0] += v_base * w1_mid * dt

            fieldY.invalidate_positions_only()

        fieldY.fieldnc[0] = pcr.poscorrector(fieldY.fieldnc[0],
fieldY.getunigrid())

        fieldY.invalidate_for_potential()


    #Drift for coherent

    if fieldY.getcohfrac() > 0:

        fieldYk=fftn(fieldY.fieldc,workers=-1)

        fieldYk *= np.exp(-chi*0.5*grids[0]*dt).astype(cts.CTYPE)

        fieldY.fieldc = ifftn(fieldYk,workers=-1).astype(cts.CTYPE)

        fieldY.invalidate_fieldc_cache()


    pot.updatepotentials(fieldY, grids)

    cleargradientcache()

    if fieldY.getcohfrac() < 1 and len(fieldY.fieldnc[0]) > 0:

        _ = fieldY.getinterpdens()

    return None
```

In this function, the potentials are updated at the end according to the new positions, to be used in the kick.

These functions are the basis for the function called in `evolver`, which is

```
def kickdrift(chi,dt,fundparameters,grids,fieldY,pot):
    kick(chi,0.5*dt,fundparameters,grids,fieldY,pot)
    drift(chi,dt,grids,fieldY,pot)
    kick(chi,0.5*dt,fundparameters,grids,fieldY,pot)
```

The last function is the object that manages the collisional events:

```
def collisionstep(evolverparameters,optparameters,grids,fieldY,dt
,gammacol2,ncollsteps):
```

First, it proceeds if there are more than one particle with

```
if len(fieldY.fieldnc[0])>=2:
```

Then, the function generates a random seed `seed = cts.ITYPE(np.random.randint(1, 2**31))` to create a compatible with Numba array with only one random element, which will be modified with the function that will be called later for the collision:

```
randarr = np.array([cts.UITYPE(seed)], dtype=cts.UITYPE)
```

Using this, the function create adaptive cells with:

```
subcells, subcellcoord, subcelllength = constructadaptivecells(fieldY, randarr)
```

which uses an auxiliary function from adaptivecells.py.

After that, it starts the collision between non-coherent elements, computing the interpolated coherent density, the interpolated non-coherent density, the sum of the weights W_1, called `totalomega` and calling the auxiliary function from collisions:

```
gncinterp = fieldY.g * fieldY.getinterpdens()
ntilde = interp.interpolatorntilde(fieldY)
totalomega = cts.FTYPE(np.sum(fieldY.weight1()))
collisionrate = col.launchercolI2(fieldY, gncinterp, ntilde, totalomega, dt,
subcells, subcellcoord, subcelllength, randarr)
```

then, it passes the obtained collision rate to gammacol2 and add a collisional step:

```
gammacol2 += collisionrate
ncollsteps  += 1
```

1.3.6 collisions.py

In this file we define functions that run the collisions. This file will use auxiliary functions defined in `additionalfunctions` and Numba. To that, the file starts with

```
import numpy as np
import math
from numba import njit
from utilities.additionalfunctions import constants as cts
from utilities.additionalfunctions import auxiliarycollisions as acol
from utilities.additionalfunctions.weights import W1
```

The first function is:

```
def launchercolI2(fieldY, gncinterp, dt, subcells, subcellcoord, subcelllength,
randarr):
```

which receives as arguments the field `fieldY`, the interpolated coherent density `gncinterp`, the time differential `dt`, the results from the adaptive cells `subcells, subcellcoord, subcelllength` and the random array for shuffling `randarr`.

Then, it defines the variables that will be passed to the internal function:

```
positions  = fieldY.fieldnc[0]

velocities = fieldY.fieldnc[1]

g = fieldY.g

Nq = (1.0 - fieldY.getcohfrac()) * fieldY.gettotalnumber()

Ntp = len(positions)

gamma = cts.FTYPE(Nq / Ntp)

Ngrid     = len(fieldY.getunigrid())

dx       = fieldY.getdx()

xmin     = np.amin(fieldY.getunigrid())

prefactor = cts.FTYPE(2.0 * g * g * Ntp / (math.pi * totalomega))
```

then, it defines a 3D array where the collision rate will be filled for each point of the space

```
collisionrate = np.zeros((Ngrid, Ngrid, Ngrid), dtype=cts.FTYPE)
```

and then it calls:

```
I2cells(subcells, subcellcoord, subcelllength, positions, velocities, gncinterp,
ntilde, dt, prefactor, gamma, randarr, collisionrate, xmin, dx, Ngrid)
```

and returns `collisionrate`.

The function `I2cells` is a simple function defined as

```
@njit(cache=True)

I2cells(subcells,subcellcoord,subcelllength,positions,velocities,gncinterp,ntilde
,dt,prefactor,gamma,randarr,collisionrate,xmin,dx,Ngrid):

    for i in range(len(subcells)):


I2subcell(subcells[i],subcellcoord[i],subcelllength[i],positions,velocities,
gncinterp,ntilde,dt,prefactor,gamma,randarr,collisionrate,xmin,dx,Ngrid)
```

which computes the colissions for each subcell using the function `I2subcell` to be defined by:

```
@njit(cache=True)

I2subcell(subcells,subcellcoord,subcelllength,positions,velocities,gncinterp,
ntilde,dt,prefactor,gamma,randarr,collisionrate,xmin,dx,Ngrid):
```

The function exits in the beginning if there is no pairs inside the subcell:

```
Numinsc = len(subcells)

if Numinsc < 2:
```

```
        return
```

Then it computes the volume of this cell:

```
dV = subcelllength * subcelllength * subcelllength
```

and it defines some quantities initialized to zero:

```
velstorer = np.empty((Numinsc, 3), dtype=velocities.dtype)
sumgnc     = 0.0
sumw1      = 0.0
sumntilde  = 0.0
```

Here `velstorer` will be an array that will contain copies of the velocities of the particles in the cell, `sumgnc` will be an object that will be the sum of the gn_c interpolated values for each particle in the cell, the same for `sumw1`, which will be the sum of the weights W_1 in the cell and `sumntilde` the sum of the interpolated values of the non-coherent density in the cell.

After this, it starts a loop over the elements in the cell, copying the velocities of the elements in `velstorer` and computing the velocity square

```
for k in range(Numinsc):
        indcel  = subcells[k]
        v0  = velocities[indcel, 0]
        v1  = velocities[indcel, 1]
        v2  = velocities[indcel, 2]
        velstorer[k, 0] = v0
        velstorer[k, 1] = v1
        velstorer[k, 2] = v2
        vsq = v0*v0 + v1*v1 + v2*v2
```

Also, the function adds the values of the intepolated densities of coherent and non-coherent, and the corresponding weight W1

```
sumgnc += gncinterp[indcel]
sumntilde += ntilde[indcel]
sumw1 += W1(vsq, gncinterp[indcel])
```

with that, that loop closes and it is computed the mean for the interpolated densities:

```
gncmean = sumgnc / cts.FTYPE(Numinsc)
ntildemean = sumntilde / cts.FTYPE(Numinsc)
```

Then, the prefactor $\frac{2\bar{g}^2}{\pi}\frac{\tilde{\bar{n}}\tilde{N}_{\rm test}}{N_{\rm cell}\Omega}$ for the probability distribution, and useful quantities for later

```
granprefactor = prefactor * ntildemean * sumw1 / cts.FTYPE(Numinsc)
invdx3 = 1.0 / (dx * dx * dx)
```

where `invdx3` is constructed to use the better speed in multiplications later.

Then, the function computes the center of the subcell in order to deposit the counts of collision rate per cell in that point:

```
midx = subcellcoord[0] + 0.5 * subcelllength
midy = subcellcoord[1] + 0.5 * subcelllength
```

```
midz = subcellcoord[2] + 0.5 * subcelllength
```

and then it translates this point to indices in the big grid:

```
fx    = (midx - xmin) / dx
fy    = (midy - xmin) / dx
fz    = (midz - xmin) / dx
ixl    = cts.ITYPE(math.floor(fx)) % Ngrid
iyl    = cts.ITYPE(math.floor(fy)) % Ngrid
izl    = cts.ITYPE(math.floor(fz)) % Ngrid
```

taking with th integer part in the indices `ix`, `iy` and `iz` the point to the left. Then, a weight is taken:

```
wxl    = fx - math.floor(fx)
wyl    = fy - math.floor(fy)
wzl    = fz - math.floor(fz)
```

and the same for the point to the right:

```
ixr = (ixl + 1) % Ngrid
iyr = (iyl + 1) % Ngrid
izr = (izl + 1) % Ngrid
wxr  = 1.0 - wxl
wyr = 1.0 - wyl
wzr = 1.0 - wzl
```

After those definitions and arrangementes the collision process begins. It starts in a loop in the elements of the cell:

```
for a in range(0, Numinsc - 1, 2):
```

where the 2 indicates that the loop jumps 2 by 2, not by one as normal. Since the elements of the cell were shuffled in `adaptivecells` when these cells were created, it takes the contiguous elements in the cell with

```
i    = subcells[a]
j    = subcells[a + 1]
```

and their velocities are computed, together with the sum of the energies $\varepsilon_i = \sqrt{\bar{v}_i^2(\bar{v}_i^2 + 4\bar{g}\bar{n}_c)}$ and the sum of their velocities:

```
vi0  = velocities[i, 0]
vi1  = velocities[i, 1]
vi2  = velocities[i, 2]
visq = vi0*vi0 + vi1*vi1 + vi2*vi2
vj0  = velocities[j, 0]
vj1  = velocities[j, 1]
vj2  = velocities[j, 2]
vjsq = vj0*vj0 + vj1*vj1 + vj2*vj2
```

```
E12  = acol.bogoliubovenergy(visq, gncmean) + acol.bogoliubovenergy(vjsq, gncmean)
v12x = vi0 + vj0
v12y = vi1 + vj1
v12z = vi2 + vj2
```

and here the auxiliary function `bogoliubovenergy` from `auxiliarycollisions` was used.

After that, the polar method of Marsaglia algorithm is used to define a direction where the particle with velocity v_3 will go out:

```
ox = oy = oz = 0.0
for attempt in range(30):
            x1 = 2.0 * acol.randomizer(randarr) - 1.0
            x2 = 2.0 * acol.randomizer(randarr) - 1.0
            r2 = x1*x1 + x2*x2
            if r2 < 1.0:
                sq = math.sqrt(1.0 - r2)
                ox = 2.0 * x1 * sq
                oy = 2.0 * x2 * sq
                oz = 1.0 - 2.0 * r2
                break
```

The code uses a rejection method, where with `for attempt in range(30)` it will try 30 times to do the process. Normally it works much before that, so that number is just an exaggerated number to ensure that the process is done. This mtehod to generate a random direction is faster than using trigonometric functions and it results in a uniform distribution of a normalized direction which module is 1.

With this, an auxiliary function is called in order to solve for the module of the velocity v_3 the equation

$$\varepsilon_1 + \varepsilon_2 - \varepsilon_3 - \varepsilon_{1+2-3} = 0$$

which represents the energy conservation of this process. Thus, the code does

```
s = acol.collenergycons(E12, v12x, v12y, v12z, ox, oy, oz, gncmean)
```

If the method in `collenergycons` gives $s = 0$, it means that there is no solution for the given equation and it means that the collision has not taken place and then it does not do nothing.

Otherwise, it computes the velocities v_3 and v_4 with their module squared:

```
v3x  = s * ox
v3y = s * oy
v3z = s * oz
v4x  = v12x - v3x
v4y  = v12y - v3y
v4z  = v12z - v3z
v3sq = s * s
```

```
v4sq = v4x*v4x + v4y*v4y + v4z*v4z
```

Then, as security check for the numerical method, the energy of the result is checked with

```
Echeck = acol.bogoliubovenergy(v3sq, gncmean) + acol.bogoliubovenergy(v4sq,
gncmean)
tol = cts.FTYPE(1.0e3) * cts.FEPS
if E12 > cts.MINVAL:
            if abs(Echeck - E12) / E12 > tol:
                continue
else:
            if abs(Echeck - E12) > tol:
                continue
```

here the tolerance to the difference of the found energy and the initial is chosen to be 10^3 times the round error of the system. If the error is bigger than this, the result is discarded.

Using the computed outgoing velocities the quantity

$$\frac{|\bar{\boldsymbol{v}}_3|^2}{W_1(\bar{v}_1)W_1(\bar{v}_2)|W_1(\bar{v}_3)|\bar{v}_3| + W_1(\bar{v}_4)|\bar{v}_4||}$$

which is part of the probability is computed:

```
W1i   = W1(visq, gncmean)
W1j   = W1(vjsq, gncmean)
W3 = W1(v3sq, gncmean)
modv4 = math.sqrt(v4sq) if v4sq > 0.0 else 0.0
W4 = W1(v4sq, gncmean)
denom = W1i * W1j * abs(W3 * s + W4 * modv4)
w = v3sq / denom
```

Then, the Bose enhancement factors are computed with an auxiliary function and with that the probability factor:

```
f3   = acol.ffunction(v3x, v3y, v3z, velstorer, gamma, dV)
f4   = acol.ffunction(v4x, v4y, v4z, velstorer, gamma, dV)
probfactor = granprefactor * w * (1.0 + f3) * (1.0 + f4)
```

The probability will be clamped to be 1 in case the computation gives a bigger number:

```
P = probfactor * dt
if P > 1.0:
            P = 1.0
```

For a small `dt` it would not give something bigger than 1, but in any case this is added.

Then, a random number is generated and if it is smaller than the probability, the velocities are updated and the collision rate is stored:

```
if acol.randomizer(randarr) < P:
            velocities[i, 0] = v3x
```

```
            velocities[i, 1] = v3y
            velocities[i, 2] = v3z
            velocities[j, 0] = v4x
            velocities[j, 1] = v4y
            velocities[j, 2] = v4z

            val = probfactor * invdx3
            collisionrate[ixl, iyl, izl] += wxr * wyr * wzr * val
            collisionrate[ixr, iyl, izl] += wxl * wyr * wzr * val
            collisionrate[ixl, iyr, izl] += wxr * wyl * wzr * val
            collisionrate[ixl, iyl, izr] += wxr * wyr * wzl * val
            collisionrate[ixr, iyr, izl] += wxl * wyl * wzr * val
            collisionrate[ixr, iyl, izr] += wxl * wyr * wzl * val
            collisionrate[ixl, iyr, izr] += wxr * wyl * wzl * val
            collisionrate[ixr, iyr, izr] += wxl * wyl * wzl * val
```

1.4 Additional functions

In the folder `additionalfunctions` we found the files that contain functions that are called from the functions in `utilities`.

1.4.1 constants.py

This is a simple file with the following lines:

```
import numpy as np


CTYPE = np.complex64
FTYPE = np.float32
ITYPE = np.int64
UITYPE = np.uint64
MINVAL  = FTYPE(np.finfo(FTYPE).tiny)
FEPS  = FTYPE(np.finfo(FTYPE).eps)
UIMAX = UITYPE(np.iinfo(UITYPE).max)
```

It defines the type of complex, float, int, uint used in the code.

Also, `MINVAL` is the minimum value that the computer can manage, `FEPS` defines the minimum distance between 1.0 and the next number that the computer can represent, and `UIMAX` is the maximum value that `UITYPE` can have.

1.4.2 parsing.py

This file contains three functions. The first is:

```
def parseargs(argv):
    path = None
    for i, arg in enumerate(argv):
        if arg == '-path':
            path = argv[i + 1]
    if path is None:
        raise ValueError("-path argument is required.")
    return path
```

This function reads the path as argument after the line `-path` from the terminal to store it for later use.

The second function is

```
def remove_comments(lines):
    new_lines = []
    for line in lines:
        if line.startswith("#"):
            continue
        line = line.split(" #")[0]
        if line.strip() != "":
            new_lines.append(line)
    return new_lines
```

which during the reading of the inifile it removes the lines starting with #, considered as comments, in order to pass the content of the inifile to `storer`.

The third function:

```
def checkerlist(param, container):
    try:
        indicestore = next(i for i, item in enumerate(container) if param in item)
        element = container[indicestore].partition("=")[2].replace(" ",
"").strip("\n")
    except StopIteration:
        print(f'Parameter {param} is missing. Please enter this parameter
 in the initial file')
        sys.exit()

    return element
```

which is used in `storer` to check if a parameter exists in the inifile, closing the program if mandatory parameters are not present and returning the value of the parameter.

1.4.3 weights.py

In this file the instructions for the weights

$$W_1 = \frac{\bar{v}_i^2 + 2\bar{g}\bar{n}_c(\bar{x})}{\sqrt{\bar{v}_i^2(\bar{v}_i^2 + 4\bar{g}\bar{n}_c(\bar{x}))}}$$

$$W_2 = \sqrt{\frac{\bar{v}_i^2}{\bar{v}_i^2 + 4\bar{g}\bar{n}_c}}$$

are defined. The functions use Numba for computation, so the fiel starts with:

```
import numpy as np
import math
from numba import njit, prange
```

and then, the first function is

```
@njit(cache=True, fastmath=True)
def W1(vsq, gnc):
    denom =  math.sqrt(vsq * (vsq + 4.0 * gnc))
    return (vsq + 2.0 * gnc) / denom
```

and the second:

```
@njit(cache=True, fastmath=True)
def W2(vsq, gnc):
    denom = vsq + 4.0 * gnc
    return math.sqrt(vsq / denom)
```

Both functions compute only one weight. The file also contains two functions which compute the same but for arrays using parallel computation:

```
@njit(cache=True, parallel=True, fastmath=True)
def weight1numba(modvelsq, densterm):
    N   = modvelsq.shape[0]
    out = np.empty(N, dtype=modvelsq.dtype)
    for i in prange(N):
        vsq    = modvelsq[i]
        gnc    = densterm[i]
        out[i] = (vsq + 2.0 * gnc) / math.sqrt(vsq * (vsq + 4.0 * gnc))
    return out
```

and

```
@njit(cache=True, parallel=True, fastmath=True)
def weight2numba(modvelsq, densterm):
    N   = modvelsq.shape[0]
```

```
    out = np.empty(N, dtype=modvelsq.dtype)
    for i in prange(N):
        vsq    = modvelsq[i]
        gnc    = densterm[i]
        out[i] = math.sqrt(vsq / (vsq + 4.0 * gnc))
    return out
```

1.4.4 densitync.py

This file contains the function used to compute the non-coherent density and the thermal contribution. For this file it is used Numba to compute in parallel the densities. To do that, we call the following Numba elements, together with the code required functions:

```
import numpy as np
from numba import njit, prange
from numba.np.ufunc import parallel as numba_parallel
from numba import get_num_threads
from utilities.additionalfunctions import constants as cts
```

The only function here is

```
@njit(parallel=True, fastmath=True)
def computedensity(pos, w1, w2, Ngrid, dx, xmin):
```

It requires as arguments the array of positions `pos`, two weights `w1` and `w2`, which are

$$W_1 = \frac{\bar{v}_i^2 + 2\bar{g}\bar{n}_c(\bar{x})}{\sqrt{\bar{v}_i^2(\bar{v}_i^2 + 4\bar{g}\bar{n}_c(\bar{x}))}}$$

$$W_2 = \sqrt{\frac{\bar{v}_i^2}{\bar{v}_i^2 + 4\bar{g}\bar{n}_c}}$$

Also, it requires the number of sites per length `Ngrid` and the space between sites `dx` and `xmin`, which is the minimum value in the grid along one axis.

In the function to obtain the number of threads for the parallel execution it is used the instruction

```
num_threads = get_num_threads()
```

and then, the fucntion stores the number of total cells in the density and the total number of test particles to create arrays to store the density and the thermal contribution:

```
dens1 = np.zeros((num_threads, total_cells), dtype=cts.FTYPE)
dens2 = np.zeros((num_threads, total_cells), dtype=cts.FTYPE)
```

Here, for each thread there will be a one-dimensional array with length equal to the number of total cells.

To map the 3D grid coordinates into the 1D local density arrays, we define memory strides:

```
sy = Ngrid
sx = Ngrid**2
```

After that, the function computes in parallel subgroups of particles with the for cycle:

```
for i in prange(Nparticles):
```

and identifying in what part of the dens arrays it will store the results via

```
thread_id = numba_parallel.get_thread_id()
```

Then, it write the positions in the coordinates:

```
rx = (pos[i, 0] - xmin) / dx
ry = (pos[i, 1] - xmin) / dx
rz = (pos[i, 2] - xmin) / dx
```

and keeps the corresponding weights with

```
p1 = w1[i]
p2 = w2[i]
```

Then, the element in the bottom-left of the cell is stored in:

```
bx = cts.ITYPE(np.floor(rx))
by = cts.ITYPE(np.floor(ry))
bz = cts.ITYPE(np.floor(rz))
```

and the corresponding fractional distance of the particle to that point in:

```
fx = rx - bx
fy = ry - by
fz = rz - bz
```

Then, using cloud-in-cell assigns the corresponding weights to each vertex of the cell and multiply it with the weights W_1 and W_2 according to if it is the density or the thermal contribution, together with the implementation of the periodic boundary conditions:

```
for ix in range(2):
            wx = fx if ix else (1.0 - fx)
            idx_x = ((bx + ix) % Ngrid) * sx

            for iy in range(2):
                wy = wx * (fy if iy else (1.0 - fy))
                idx_y = ((by + iy) % Ngrid) * sy

                for iz in range(2):
                    wz = wy * (fz if iz else (1.0 - fz))
                    idx_z = (bz + iz) % Ngrid

                    flat_idx = idx_x + idx_y + idx_z
```

```
                    dens1[thread_id, flat_idx] += wz * p1
                    dens2[thread_id, flat_idx] += wz * p2
```

After that, it reshape the arrays to the 3D coordinates that are returned:

```
shape3d=(Ngrid,Ngrid,Ngrid)
d1 = np.sum(dens1, axis=0).reshape(shape3d)
d2 = np.sum(dens2, axis=0).reshape(shape3d)
return d1, d2
```

1.4.5 interpolations.py

This file contains the functions that are used to interpolate the values of the density or potential from grid points to point where the non-coherent particles are located.

To do that, Numba will be used and with that the function requires the calling of:

```
import numpy as np
from numba import njit, prange
from utilities.additionalfunctions import constants as cts
```

Then, the first function is constructed:

```
@njit(parallel=True, fastmath=True)
def interpolatescalar(pos, scalararray, Ngrid, dx, xmin):
```

which requires as arguments the array of positions `pos`, the array to be interpolated `scalararray`. Also, it requires the number of sites per length `Ngrid` and the space between sites `dx` and `xmin`, which is the minimum value in the grid along one axis.

Similarly to the case of `densitync.py`, it is computed in parallel with the for cycle:

```
for i in prange(n_particles):
```

and here it is not required `num-threads` as in the density computation, since each particle writes to a unique index in the array which stores the result, avoiding any memory race conditions.

Inside the cycle, similarly to the density case, it is defined the coordinates, the bottom-left position of the cell, and the corresponding fractional distance of the particle to that point with

```
rx = (pos[i, 0] - xmin) / dx
ry = (pos[i, 1] - xmin) / dx
rz = (pos[i, 2] - xmin) / dx


bx, by, bz = cts.ITYPE(np.floor(rx)), cts.ITYPE(np.floor(ry)),
cts.ITYPE(np.floor(rz))
fx, fy, fz = rx - bx, ry - by, rz - bz
```

Then, we define a variable to store the values for each particle:

```
accumula = cts.FTYPE(0.0)
```

and then, cloud-in-cell is used to assign from the values in the grid to the considered point:

```
for ix in range(2):
            wx = fx if ix else (1.0 - fx)
```

```
            idx_x = (bx + ix) % Ngrid

            for iy in range(2):
                wy = wx * (fy if iy else (1.0 - fy))
                idx_y = (by + iy) % Ngrid

                for iz in range(2):
                    wz = wy * (fz if iz else (1.0 - fz))
                    idx_z = (bz + iz) % Ngrid

                    accumula += wz * scalararray[idx_x, idx_y, idx_z]
```

Then the result is returned, completing the function.

A similar function is defined to interpolate vectorial elements:

```
@njit(parallel=True, fastmath=True)
def interpolatevector(pos, vecarray, Ngrid, dx, xmin):
```

the only difference with `interpolatescalar` is that inside the loop

```
for i in prange(Nparticles):
```

it is defined another loop for each component:

```
for l in range(3):
    veccomponent = vecarray[l]
```

and then the process is the same.

Finally, two functions are constructed to use `interpolatescalar` to obtain the density and the potential:

```
def interpolatordens(fieldY):
    dx = fieldY.getdx()
    pos = fieldY.fieldnc[0]
    unigrid = fieldY.getunigrid()
    xmin = np.amin(unigrid)
    Ngrid = len(unigrid)
    dens = np.ascontiguousarray(fieldY.densityc, dtype=cts.FTYPE)

    return interpolatescalar(pos, dens, Ngrid, dx, xmin)

def interpolatorpot(fieldY,pot):
    dx = fieldY.getdx()
    pos = fieldY.fieldnc[0]
```

```
    unigrid = fieldY.getunigrid()
    xmin = np.amin(unigrid)
    Ngrid = len(unigrid)
    pote = np.ascontiguousarray(pot.mfpotential, dtype=cts.FTYPE)

    return interpolatescalar(pos, pote, Ngrid, dx, xmin)
```

where in both functions `np.ascontiguousarray` is used to ensure fast computation in the passing to Numba.

1.4.6 storing.py

This file contains only one function, which is used to save the data from the simulation in the disk. The function is

```
def savingstep(indexstep, t, fundparameters, evolverparameters, fieldY, timeline, gammacol2, ncollsteps):
```

The function first store the filepath where the inifile is to save the results:

```
filepath=fundparameters[4]
```

and then it stores the current simulation time in the array `timeline`: `timeline[indexstep]=t`

Then, it saves in the folder evolution inside this filepath, which was created (if it did not exist) in the main at the beginning, the arrays `fieldc` and `fieldnc`:

```
np.save(f'{filepath}'+'/evolution/fieldc_'+str(indexstep)+'.npy', fieldY.fieldc)
np.save(f'{filepath}'+'/evolution/fieldnc_'+str(indexstep)+'.npy', fieldY.fieldnc)
```

in the last step of evolution it saves the array with the time and collisions:

```
if indexstep==evolverparameters[0] // evolverparameters[1]:
        np.save(f'{filepath}'+'/timeline.npy', timeline)
        np.save(f'{filepath}'+'/gammacol2.npy', gammacol2 / max(ncollsteps, 1))
```

1.4.7 acceleration.py

In this file we have all the elements needed to compute the acceleration of the test particles in the evolution. To do that, Numba is used. Therefore, the file starts with

```
import numpy as np
from utilities.additionalfunctions import constants as cts
from utilities.additionalfunctions import interpolations as inter
from numba import njit, prange
```

The first function is a Numba based computer of the gradient of a function in the grid using centered finite difference:

```
@njit(parallel=True, fastmath=True)
def gradient3d(f, dx):
```

Here, the arguments of the function are the function in the grid `f` and the distance between sites `dx`.

We start the function extracting the dimensions of the function:

```
nx, ny, nz = f.shape
```

Since the method uses

$$f'(x) \approx \frac{f(x+\Delta x) - f(x-\Delta x)}{2\Delta x}$$

we define the object $1/(2\Delta x)$ in

```
inv2dx = cts.FTYPE(1.0 / (2.0 * dx))
```

and then, we construct the arrays for each component of the gradient:

```
gx = np.empty((nx, ny, nz), dtype=cts.FTYPE)
gy = np.empty((nx, ny, nz), dtype=cts.FTYPE)
gz = np.empty((nx, ny, nz), dtype=cts.FTYPE)
```

Then, we start a parallel computation for particles in the axis x:

```
for i in prange(nx):
        inext = (i + 1) % nx
        iprev = (i - 1) % nx
```

where in `inext` and `iprev` we extract the indices of the next and previous neighbor respectively, and imposing periodic boundary conditions.

Then, for each i it starts another for in the axis y:

```
for j in range(ny):
            jnext = (j + 1) % ny
            jprev = (j - 1) % ny
            f_ij  = f[i,  j,  :]
            f_inextj = f[inext, j,  :]
            f_iprevj = f[iprev, j,  :]
            f_ijnext = f[i,  jnext, :]
            f_ijprev = f[i,  jprev, :]

            gx[i, j, 0] = (f_inextj[0]  - f_iprevj[0] ) * inv2dx
            gy[i, j, 0] = (f_ijnext[0]  - f_ijprev[0] ) * inv2dx
            gz[i, j, 0] = (f_ij [1]  - f_ij [nz-1]) * inv2dx
```

where the extraction of slices with `f_ij  = f[i,  j,  :]` and the next lines maximises the eficiency of cache.

After this function we define in the file an element of cache as a dictionary:

```
gradcache = {}
```

It is an empty dictionary that will contain the key `cache_key` and as values the arrays `gx`, `gy` and `gz` computed from `gradient3d`.

After this definition the second function is defined:

```
def gradientcached(arr, dx, cache_key):
```

which requires as arguments the object to take the gradient `arr`, the space between sites `dx` and a label `cache_key` which is the key of `gradcache`.

The first action of this function is extract from the dictionary the value for the indicated key:

```
entry = gradcache.get(cache_key)
```

then, if there is entry it returns it with

```
if entry is not None:
        return entry
```

if it does not exist, the code continue computing the gradient with

```
grad = gradient3d(arr, dx)
```

and storing the key and the value of the computed gradient with

```
gradcache[cache_key] = grad
```

After it, the cache is cleaned if there are more than 4 stored gradients:

```
if len(gradcache) > 4:
        oldest = next(iter(gradcache))
        del gradcache[oldest]
```

and finally it returns the computed gradient if it was computed.

Another function related to the cache for the gradient computation is

```
def cleargradientcache():
    gradcache.clear()
```

which simply clear the dictionary `gradcache`.

The last function is the one that computes the acceleration for the non-coherent elements using the gradient:

```
def acceleration(grids, fieldY, pot):
```

The required arguments are the grid, the field used in the simulation and the defined potential.

First, the function defines the useful quantities:

```
dx          = fieldY.getdx()
xmin        = np.amin(fieldY.getunigrid())
Ngrid       = len(fieldY.getunigrid())
pos         = fieldY.fieldnc[0]
```

Them it defines the keys for the cache:

```
key_q  = ('q',  id(pot.potentialq), pot.potversion)
key_mf = ('mf', id(pot.mfpotential), pot.potversion)
```

The three elements of the key ensure that the returned element corresponds to the required. The first element corresponds to a name to distinguish between `potentialq` and `potentialmf`. The second corresponds to the location in the RAM memory, and the third refers to the version of the updated potential.

Then, it calls to the function `gradientcached` to compute the gradients of the potentials:

```
dU_q  = gradientcached(np.ascontiguousarray(pot.potentialq,  cts.FTYPE), dx,
key_q)
dU_mf = gradientcached(np.ascontiguousarray(pot.mfpotential, cts.FTYPE), dx,
key_mf)
```

and with that, the accelerations via `interpolatevector`:

```
acel_q  = -inter.interpolatevector(pos, dU_q,  Ngrid, dx, xmin)
acel_mf = -inter.interpolatevector(pos, dU_mf, Ngrid, dx, xmin)
```

and then, returning those values.

1.4.8 poscorrector.py

This file contains the function that implements the periodic boundary conditions after the evolution that can take out of the box non-coherent elements. It uses Numba:

```
import numpy as np
from numba import njit, prange
```

and we define the function:

```
@njit(parallel=True, fastmath=True)
def poscorrectornumba(x, Lmin, Lmax, grid_spacing):
```

In the function it is defined the length of the box:

```
L = (Lmax - Lmin) + grid_spacing
```

and then with numba we parallelize for the total of particles and compute the correction to coordinate:

```
Nparticles = x.shape[0]
for i in prange(Nparticles):
    x[i, 0] = Lmin + ((x[i, 0] - Lmin) % L)
    x[i, 1] = Lmin + ((x[i, 1] - Lmin) % L)
    x[i, 2] = Lmin + ((x[i, 2] - Lmin) % L)
```

and we return `x`.

Also, in the file it is defined a wrapper to be called in the simulation, that calls internally `poscorrectornumba`:

```
def poscorrector(x,grid):
    dx = grid[1] - grid[0]
    return poscorrectornumba(x, np.amin(grid), np.amax(grid), dx)
```

1.4.9 adaptivecells.py

In this file we construct the functions to generate collision cells which will be used for binning the particles that will participate in a collision. The idea is to split first in a master grid consisting on cuboids of equal size, then, those cells are divided into smaller cells depending on the particle number in the cell. To do this, Numba will be used.

Thus, the file starts with:

```
import numpy as np
import math
from numba import njit
from numba.typed import List
from utilities.additionalfunctions import constants as cts
from utilities.additionalfunctions import auxiliarycollisions as acol
```

The first function is:

```
def constructadaptivecells(fieldY, randarr, Nth=200):
```

where the arguments are the field `fieldY` and the threshold value `Nth` which defines the average number of particles that every collision cell must contain. By default it will be chosen 200, but it could be modified.

The function first defines the number of division per axis for the construction of the master cells

```
Nm = max(1, len(fieldY.getunigrid()) // 4)
```

The number is bigger than the subdivisions in the grid to avoid a big number of cells if they are not required. These cells will be subdivided if they have a large number of particles. Thus, as starting point it is fine.

Then it stores the values used to the creation of the cells:

```
positions = fieldY.fieldnc[0]
dx    = fieldY.getdx()
xmin    = np.amin(fieldY.getunigrid())
xmax    = np.amax(fieldY.getunigrid()) + dx
```

then it creates three arrays with

```
subcells, subcellcoord, subcelllength = buildcollisioncells(positions, xmin, xmax,
Nm, Nth, randarr)
```

with the function `buildcollisioncells` constructed in this file. Then, these three objects are returned.

The next three functions are auxiliary functions for `buildcollisioncells`. The first two of them are exactly the same, except that one computes in parallel and the other not. The reason behind that is that for a huge number of particles the parallel computation accelerates the computation, however for a small number, parallelizing could be overhead. And both are grouped in one wrapper that lauches one of them according to the required case.

Thus, we define first:

```
@njit(cache=True, parallel=True)
def cellidsparallel(positions, indices, ox, oy, oz, nx, ny, nz, dx, dy, dz):
```

which creates an array that will store the number of cell associated to each test particle. It receives as arguments the position array of the particles `positions`, an array containing the indices of the positions of the particles `indices`, the coordinates of the origin of coordinates of the cell `ox`, `oy`, `oz`, the number of spaces per axis of each cell `nx`, `ny`, `nz` and the length in the spaces `dx`, `dy` and `dz`.

The function create an empty array of dimensions of the particles in the cell to be filled:

```
N       = indices.shape[0]
```

```
cellid = np.empty(N, dtype=cts.ITYPE)
```

and then it is filled with the index corresponding to that cell with

```
for k in prange(N):
        i = indices[k]
        lx = cts.ITYPE((positions[i, 0] - ox) / dx)
        ly = cts.ITYPE((positions[i, 1] - oy) / dy)
        lz = cts.ITYPE((positions[i, 2] - oz) / dz)
        lx = min(max(lx, cts.ITYPE(0)), nx - cts.ITYPE(1))
        ly = min(max(ly, cts.ITYPE(0)), ny - cts.ITYPE(1))
        lz = min(max(lz, cts.ITYPE(0)), nz - cts.ITYPE(1))
        cellid[k] = lx * ny * nz + ly * nz + lz
```

Then it returns the array `cellid`.

The analogue function is

```
@njit(cache=True)
def cellidsserie(positions, indices, ox, oy, oz, nx, ny, nz, dx, dy, dz):
```

where the only difference is that instead of `prange` it uses `range`.

The third function calls one of the `cellids`:

```
@njit(cache=True)
def computecellids(positions, indices, ox, oy, oz, nx, ny, nz, dx, dy, dz):
    if indices.shape[0] >= cts.ITYPE(5000):
        return cellidsparallel(positions, indices, ox, oy, oz, nx, ny, nz, dx, dy,
dz)
    return cellidsserie(positions, indices, ox, oy, oz, nx, ny, nz, dx, dy, dz)
```

Estimating that over 5000 particles per cell parallelizing is the optimal situation.

The last function is, which construct the collision cells is:

```
@njit(cache=True)
def buildcollisioncells(positions, xmin, xmax, Nm, Nth, randarr):
```

First it defines the space between points of the master cell with

```
Ntp = positions.shape[0]
L = xmax - xmin
masterdx = L / cts.FTYPE(Nm)
allindices = np.arange(Ntp, dtype=cts.ITYPE)
```

which is passed to the function `computecellids` to generate the array with the number of cell for each particle:

```
cellid = computecellids(positions, allindices, xmin, xmin, xmin, Nm, Nm, Nm,
masterdx, masterdx, masterdx)
```

Now, the code allocates a contiguous block of memory

```
arrayparttogether = np.empty(Ntp, dtype=cts.ITYPE)
```

to store the reordered particle indices in such a way that the particles in the cell zero, will occupy the firsts entries in the array, then, contiguous, the particles of the cell one will fill the next spaces and so on.

After that, it computes the total number of master cells with

```
Nm3 = Nm * Nm * Nm
```

and then, it creates an array of dimension `Nm3+1` to define the starting and end point where the particles of the cell in `arrayparttogether` are:

```
segmenter = np.zeros(Nm3 + 1, dtype=cts.ITYPE)
```

this segmentation works with the loops:

```
for i in range(Ntp):
        segmenter[cellid[i] + 1] += 1
for j in range(Nm3):
        segmenter[j + 1] += segmenter[j]
```

The first loop counts the number of particles per cell and it adds to the segmenter array in such a way that the cell zero will be in the entry `segmenter[1]`, the cell one in `segmenter[2]` and so on. Thus, the first entry of `segmenter` is zero since it defines the starting point of indices for `arrayparttogether`. The loop in `j` adds in `segmenter` the values of the previous elements to define the indices where the cells start and end. It means that cell zero starts in the index given by `segmenter[0]` and it ends in the index given by `segmenter[1]`.

Then, a new array is defined:

```
occupationcounter = np.zeros(Nm3, dtype=cts.ITYPE)
```

which counts how many particles have been added per cell in the array `arrayparttogether`. Then, it is filled with

```
for i in range(Ntp):
        c = cellid[i]
        arrayparttogether[segmenter[c] + occupationcounter[c]] = i
        occupationcounter[c] += 1
```

with `segmenter[c]` we obtain the value of the starting of the cell, and with `occupationcounter[c]` we tell how many spaces add to the value given by `segmenter[c]`, since they are occupied by other particles.

After this, Numba lists are created, which will give the results from the divisions of the master cells:

```
subcells = List()
subcellcoord = List()
subcelllength = List()
```

The first list will contain arrays of indices of particles, the second the coordinate of the corner of the subcells and the third will contain the length of the subcell.

Now, the function will work in each cell. For that enters in a for loop and computes the number of elementes in that cell, and it does not anything if the cell does not have particles:

```
for indmc in range(Nm3):
        Nmc = segmenter[indmc + 1] - segmenter[indmc]
        if Nmc == 0:
            continue
```

Then, we construct the «zeros» of each master cell, to do that, we construct the indices of each cell m_x, m_y and m_z and then we use them for the coordinates of the border of each cell:

```
mz = indmc % Nm

my = (indmc // Nm) % Nm

mx = indmc // (Nm * Nm)

cx = xmin + mx * masterdx

cy = xmin + my * masterdx

cz = xmin + mz * masterdx
```

After that, we define if the cell requires more splitting:

```
if Nmc <= Nth:
            l = 0
else:
            l = cts.ITYPE(math.floor(math.log2(cts.FTYPE(Nmc) / cts.FTYPE(Nth))))
```

If the number of elements in the master cell is smaller than the threshold `Nth`, then $l = 0$ and there will be no splitting. Otherwise, it is defined as the floor of $\log_2\left(\frac{N_{\mathrm{mc}}}{N_{\mathrm{th}}}\right)$, giving the number of refinements to that cell. In other words, it is the number of possible subdivisions.

Then, the function will split a number of times per axis defined by

```
sx = cts.ITYPE(1) << cts.ITYPE((l + 2) // 3)

sy = cts.ITYPE(1) << cts.ITYPE((l + 1) // 3)

sz = cts.ITYPE(1) << cts.ITYPE(l       // 3)
```

and each master cell length is divided by the number of cuts giving the length of each small cell:

```
subdx   = masterdx / cts.FTYPE(sx)

subdy   = masterdx / cts.FTYPE(sy)

subdz   = masterdx / cts.FTYPE(sz)
```

Then, we choose as the side of the effective subcell as the minimum of the three lengths defined:

```
subside = min(subdx, min(subdy, subdz))
```

and the array containing the indices of the position of the elements in the current master cell, which are extracted with

```
subcellindices = arrayparttogether[segmenter[indmc]:segmenter[indmc] + Nmc]
```

This is the local version of the array `allindices` defined for the master cells. Also, it is constructed the local version of `cellid` for the subcells with

```
subcellid = computecellids(positions, subcellindices, cx, cy, cz, sx, sy, sz,
subdx, subdy, subdz)
```

After that, it is computed the total number of subcells in that master cell:

```
numbersubcells = sx * sy * sz
```

to create

```
subsegmenter = np.zeros(numbersubcells + 1, dtype=cts.ITYPE)
```

and to do the same process for the subcells:

```
for k in range(Nmc):
```

```
            subsegmenter[subcellid[k] + 1] += 1
for s in range(numbersubcells):
            subsegmenter[s + 1] += subsegmenter[s]


suboccupationcounter = np.zeros(numbersubcells, dtype=cts.ITYPE)
miniarrayparts   = np.empty(Nmc, dtype=cts.ITYPE)
for k in range(Nmc):
            sid = subcellid[k]
            miniarrayparts[subsegmenter[sid] + suboccupationcounter[sid]] =
subcellindices[k]
            suboccupationcounter[sid] += 1
```

Here the difference with the global process is the creation of the array `miniarrayparts`, which contains the indices of the particles in each subcell.

Similarly to the case of the master cell, it enters in a loop of the subcells and takes the number of elements and goes out of the loop if there is no particles in such subcell:

```
for s in range(numbersubcells):
            Ns = subsegmenter[s + 1] - subsegmenter[s]
            if Ns == 0:
                continue
```

Then, it extracts the array with the indices of particles in that subcell:

```
subcellarr  = miniarrayparts[subsegmenter[s]:subsegmenter[s] + Ns].copy()
```

It computes the coordinates of the subcell with a similar method used in the master cell:

```
szcoord = s % sz
sycoord = (s // sz) % sy
sxcoord = s // (sy * sz)
```

and it stores them in an array

```
subcellcorner = np.array([cx + sxcoord * subdx, cy + sycoord * subdy, cz + szcoord
* subdz], dtype=positions.dtype)
```

In this point, the particles inside the cell are shuffled to do a random pairing with

```
for i in range(Ns - 1, 0, -1):
                j = cts.ITYPE(acol.randomizer(randarr) * cts.FTYPE(i + 1))
                tmp            = subcellarr[i]
                subcellarr[i] = subcellarr[j]
                subcellarr[j] = tmp
```

where here an auxiliary function `randomizer` from `auxiliarycollisions` was used.

After this, the function uses Transient Adaptive Sub-Cells (TASC) to process the collisions in smaller cells than the subcells. To do that, we will split the subcell in parts that we can found an average of 3 particles. Thus, we divide the total number in the cell by 3 by axis and we define the length of these subdivisions:

```
nt      = max(cts.ITYPE(1), cts.ITYPE(math.floor((Ns / 3.0) ** (1.0 / 3.0))))
tascdx  = subside / cts.FTYPE(nt)
```

And as before, another call to `cellidserie` is used to define the array with the indices of the objects in subcells:

```
tascid = cellidsserie(positions, subcellarr, subcellcorner[0], subcellcorner[1],
subcellcorner[2], nt, nt, nt, tascdx, tascdx, tascdx)
```

Then, the function computes the highest index from `tascid`:

```
counteroftasc = cts.ITYPE(1)
for i in range(Ns):
        if tascid[i] + cts.ITYPE(1) > counteroftasc:
                counteroftasc = tascid[i] + cts.ITYPE(1)
```

and it uses this number to create an array with the information of how many particles per block we found:

```
tasccounts  = np.zeros(counteroftasc, dtype=cts.ITYPE)
for i in range(Ns):
                tasccounts[tascid[i]] += cts.ITYPE(1)
```

With this information, an analogue to `segmenter` and `subsegmenter` is created:

```
tascsegmenter = np.zeros(ntasc + cts.ITYPE(1), dtype=cts.ITYPE)
for i in range(counteroftasc):
                tascsegmenter[i + cts.ITYPE(1)] = tascsegmenter[i] + tasccounts[i]
```

Now, similarly to the case of `miniarrayparts` and `suboccupationcounter` it is defined

```
tascoccupationcounter = np.zeros(counteroftasc, dtype=cts.ITYPE)
tascarray             = np.empty(Ns, dtype=cts.ITYPE)
for i in range(Ns):
        tid = tascid[i]
        tascarray[tascsegmenter[tid] + tascoccupationcounter[tid]] = subcellarr[i]
        tascoccupationcounter[tid] += cts.ITYPE(1)
```

Then for each subdivision it takes the number of elements and if there is less than two elements goes out of the loop:

```
for t in range(counteroftasc):
                Nt = tascsegmenter[t + cts.ITYPE(1)] - tascsegmenter[t]
                if Nt < cts.ITYPE(2):
                    continue
```

Otherwise, it extracts the array with the indices of particles in that subdivision:

```
tascarr = tascarray[tascsegmenter[t]:tascsegmenter[t] + Nt].copy()
```

Finally it adds the corresponding arrays to the lists:

```
subcells.append(tascarr)
```

```
subcellcoord.append(subcellcorner)
subcelllength.append(subside)
```

When the loops ends the function ends returning `subcells`, `subcellcoord` and `subcelllength`.

1.4.10 auxiliarycollisions.py

This file contains some useful functions called during the collisional processes. Here Numba is used and also the weights are required. Therefore, the file starts with:

```
import numpy as np
import math
from numba import njit
from utilities.additionalfunctions import constants as cts
from utilities.additionalfunctions.weights import W1, W2
```

The first function is a Marsaglia Xorshift random generator number

```
@njit(cache=True)
def randomizer(rs):
    x = rs[0]
    x ^= x << cts.UITYPE(13)
    x ^= x >> cts.UITYPE(7)
    x ^= x << cts.UITYPE(17)
    rs[0] = x
    return cts.FTYPE(x) / cts.FTYPE(cts.UIMAX)
```

This function receives an array of one element as argument, which contains the seed for the random number generation. The function stores this seed in `x = rs[0]`. Then, `x ^= x << cts.UITYPE(13)` moves the bits 13 positions to the left, and it does a XOR with the original value. Then, with `x ^= x >> cts.UITYPE(7)` it moves to the right 7 positions and it does a XOR with the previous value. Finally, with `x ^= x << cts.UITYPE(17)` it moves 17 positions to the right and it does a XOR again. This numbers $(13, 7, 17)$ generate a sequence which takes too long time to be repeated serving as a good random number. Finally the function stores as a new seed this number and it returns as random number this value divided by the maximum value that the constant `UITYPE` can have ensuring that the number always lies between 0 and 1.

The second function corresponds to the computation of

$$\varepsilon_i = \sqrt{\bar{v}_i^2(\bar{v}_i^2 + 4\bar{g}\bar{n}_c)}$$

using Numba:

```
@njit(cache=True)
def bogoliubovenergy(vsq, gnc):
    arg = vsq * (vsq + 4.0 * gnc)
    return math.sqrt(arg)
```

The third function computes the estimated function f for the considered particle in the bin:

```
@njit(cache=True)
def ffunction(vx, vy, vz, velstorer, gamma, dV, Nv=8):
```

It receives as arguments, the three components of the new velocity for the particle `vx`, `vy` and `vz`, the array containing the components of the velocity of the subcell `velstorer`, the quantity $\gamma = \frac{(1-f)N_{\text{tot}}}{\tilde{N}_{\text{test}}}$ in `gamma`, the volume differential of the subcell `dV` and thenumber of bin divisions by axis `Nv` which by default was chosen to be 8.

The function defines the variable that defines the maximum component of velocity in all the axes and it fills it:

```
vmax = 0.0
for k in range(velstorer.shape[0]):
        for d in range(3):
            av = abs(velstorer[k, d])
            if av > vmax:
                vmax = av
```

Then, it also compares this maximum with the components `vx`, `vy` and `vz`:

```
av = abs(vx) if abs(vx) > abs(vy) else abs(vy)
av = av if av > abs(vz) else abs(vz)
if av > vmax:
        vmax = av
```

Then, the velocity distance between the bins is obtained dividing the range in the number of bins and obtaining the phase space differential:

```
dv = 2.0 * vmax / cts.FTYPE(Nv)
dVphase = dV * dv * dv * dv
invdv   = 1.0 / dv
```

Also, the quantity `invdv` is defined for the speed of the computation.

After that, it defines the number of bin in which the particle is:

```
bx = min(max(cts.ITYPE((vx + vmax) * invdv), 0), Nv - 1)
by = min(max(cts.ITYPE((vy + vmax) * invdv), 0), Nv - 1)
bz = min(max(cts.ITYPE((vz + vmax) * invdv), 0), Nv - 1)
```

Then, we count each element in the subcell if they are in the same bin:

```
count = 0
for k in range(velstorer.shape[0]):
        kx = min(max(cts.ITYPE((velstorer[k, 0] + vmax) * invdv), 0), Nv - 1)
        ky = min(max(cts.ITYPE((velstorer[k, 1] + vmax) * invdv), 0), Nv - 1)
        kz = min(max(cts.ITYPE((velstorer[k, 2] + vmax) * invdv), 0), Nv - 1)
        if kx == bx and ky == by and kz == bz:
            count += 1
```

then, the estimated value of the function f is the count multiplied by γ and divided by the phase space volume:

```
return cts.FTYPE(count) * gamma / dVphase
```

which is returned.

The last function in the file computes the value of the module of velocity for the outgoing particle $v3$ using the energy conservation:

$$\varepsilon(\bar{v}_1)+\varepsilon(\bar{v}_2)-\varepsilon(\bar{v}_3)-\varepsilon(\bar{v}_4)=0$$

with $\bar{v}_1+\bar{v}_2+\bar{v}_3+\bar{v}_4=0$.

This function is

```
@njit(cache=True)
```

```
def collenergycons(E12, v12x, v12y, v12z, ox, oy, oz, gnc):
```

where the arguments of the functions are the sum of the incoming energies $\varepsilon(\bar{v}_1)+\varepsilon(\bar{v}_2)$ `E12`, the components of the incoming velocities $\bar{v}_1+\bar{v}_2$ `v12x`, `v12y` and `v12z`, the components of the chosen direction of outgoing particles `ox`, `oy` and `oz`, and the product of the self coupling with the average interpolated density of the cell gn_c `gnc`.

The function defines a maximum number of iterations in the method and a tolerance:

```
maxiterations=20
```

```
tol=1.0e-8
```

Also, the module square of $\bar{v}_1+\bar{v}_2$ is computed together with the dot product with the outgoing direction for later use:

```
v12sq  = v12x*v12x + v12y*v12y + v12z*v12z
```

```
v12dot = v12x*ox + v12y*oy + v12z*oz
```

Then, an inline function definition is constructed, which computes the quantities used to the Newton-Raphson method:

```
def solvereq(s):
```

this sub-function computes the square of the velocity `s` and the module squared of the outgoing particle $\bar{v}_4=\bar{v}_1+\bar{v}_2-\bar{v}_3$:

```
s2   = s * s
```

```
v4sq = v12sq - 2.0 * s * v12dot + s2
```

In case that for some reason this last value results in negative due to numerics, it is set to zero:

```
if v4sq < 0.0:
```

```
            v4sq = 0.0
```

Then, the corresponding energies are computed:

```
arg3 = s2 * (s2 + 4.0 * gnc)
```

```
arg4 = v4sq * (v4sq + 4.0 * gnc)
```

```
if arg3 <= 0.0 or arg4 <= 0.0:
```

```
            return -E12, 0.0
```

```
ene3 = math.sqrt(arg3)
```

```
ene4 = math.sqrt(arg4)
```

and then, the function to be solved:

```
F = ene3 + ene4 - E12
```

Then, the derivative of `ene3` is computed respect to s: $\frac{d\varepsilon_3}{ds} = \frac{2s^2 + 4\bar{g}\bar{n}_c}{\varepsilon_3} s$

```
dene3ds = ((2.0*s2   + 4.0 * gnc) / ene3) * s
```

the derivative of $\bar{v}_4^2$ respect to s is also computed:

```
dv4sq = 2.0 * s - 2.0 * v12dot
```

and the derivative of `ene4` respect to s: $\frac{d\varepsilon_4}{ds} = \frac{\bar{v}_4^2 + 2\bar{g}\bar{n}_c}{\varepsilon_4} \frac{d\bar{v}_4^2}{ds}$

```
dene4ds = ((v4sq + 2.0 * gnc) / ene4)  * dv4sq
```

and returning those values

```
return F, dene3ds + dene4ds
```

After this definition of function, the code will analyse the possibility of $\bar{v}_4 = 0$, which means that the other particle took all the momentum. In this case, the outcoming energy is

$$\varepsilon = \sqrt{(\bar{v}_1 + \bar{v}_2)^2((\bar{v}_1 + \bar{v}_2)^2 + 4\bar{g}\bar{n}_c)} > (\bar{v}_1 + \bar{v}_2)^2$$

and we compare it with the initial energy:

```
arg0 = v12sq * (v12sq + 4.0 * gnc)
F0   = (math.sqrt(arg0) if arg0 > 0.0 else 0.0) - E12
```

If `F0` is smaller than zero, then $E_{12} > \sqrt{(\bar{v}_1 + \bar{v}_2)^2((\bar{v}_1 + \bar{v}_2)^2 + 4\bar{g}\bar{n}_c)} > (\bar{v}_1 + \bar{v}_2)^2$. It means that there is no classical solution for $\bar{v}_3$ and $\bar{v}_4$ that adding both gives $\bar{v}_1 + \bar{v}_2$ and the initial energy E_{12}. This is a forbbiden case for classical energies. On the other hand, if it is bigger or equal to zero, in principle it could be possible to have a classical case, so the code will compute the classical case

$$\bar{v}_3^2 + \bar{v}_4^2 = E_{12}$$

since it is cheaper:

```
if F0 >= 0.0:
        disc = v12dot*v12dot - 2.0*(v12sq - E12)
        if disc >= 0.0:
            s = 0.5 * (v12dot + math.sqrt(disc))
            return s
```

If it is not possible to compute a classical result, the code compute the Bogoliubov case in full. To do that, it enters in the next steps. First it defines an upper bound to start a bisection method. It is defined to be

```
shigh = math.sqrt(E12) + math.sqrt(v12sq) + 1.0
```

Then, it iterates to adjust the upper bound of the search:

```
for _ in range(15):
        Fhigh, _ = solvereq(shigh)
        if Fhigh > 0.0:
            break
```

```
        shigh *= 2.0
```

then it uses bisection to find a value for velocity, which will be the starting point for Newton-Raphson if it is far from the tolerance limit:

```
slow  = 0.0
smid = 0.5 * (slo + shigh)
for _ in range(15):
        Fmid, _ = solvereq(smid)
        if Fmid < 0.0:
            slow = smid
        else:
            shigh = smid
        smid = 0.5 * (slow + shigh)
        if shigh - slow < tol * (1.0 + smid):
            return max(smid, 0.0)
s = smid
```

Now, this value is used in Newton Raphson to find the value:

```
for _ in range(maxiterations):
        Fs, dFs = solvereq(s)
        if abs(dFs) < 1.0e-30:
            break
        snew = s - Fs / dFs
        if snew < slow:
            snew = 0.5 * (slow + s)
        if s_new > shigh:
            snew = 0.5 * (s + shigh)
        if abs(snew - s) < tol * (1.0 + s):
            return max(snew, 0.0)
        s = snew

    return max(s, 0.0)
```